Troubleshooting Slow PySpark Query in Production

Steps to Troubleshoot the Issue with Real-Time Examples

1. Execution Plan: Use .explain(True) to Detect Shuffles and Skew

- Check the execution plan to identify costly operations like **shuffles, joins, and filters**.

Example:

```
spark.sql("SELECT * FROM large_table JOIN small_table ON large_table.id = small_table.id").explain(True)
```

- If **Shuffle Hash Join** appears, consider using **Broadcast Join** for small tables.

2. Data Volume: Increase Partitions if Production Data is Larger

- Production datasets are typically **much larger** than development.
- Check data size and adjust partitions accordingly.

Example:

```
print(f"Development Data Count: {df_dev.count()}")

print(f"Production Data Count: {df_prod.count()}")
```

- Increase partitions if production data is **10x larger**:

df = df.repartition(100) # Increase partitions to distribute load.

3. Partitioning: Use .repartition(n) or .coalesce(n) Wisely

- **Repartition** redistributes data evenly (use for large shuffles).
- **Coalesce** reduces the number of partitions without full shuffle.

Example:

df = df.repartition(200) # Better parallelism

For optimized writes:

df.write.mode("overwrite").partitionBy("date").parquet("hdfs://output_folder")

4. Optimize Joins: Use Broadcast Joins for Small Tables

- If **one table is small**, **broadcast it** to avoid expensive shuffles.

Example:

from pyspark.sql.functions import broadcast

```
large_df = spark.read.parquet("hdfs://large_table.parquet")
small_df = spark.read.parquet("hdfs://small_table.parquet")

joined_df = large_df.join(broadcast(small_df), "id", "inner")
```

- This improves performance by reducing shuffle time.

5. Memory: Increase Executor Memory and Shuffle Partitions

- If queries **run out of memory**, increase executor memory.

Example:

```
spark = SparkSession.builder \
    .config("spark.executor.memory", "8g") \
    .config("spark.executor.cores", "4") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()
```

6. Data Skew: Identify Skew and Apply Salting

- If **certain keys have too much data**, the query will be slow.

Identify Skew:

```
df.groupBy("customer_id").count().orderBy("count", ascending=False).show()
```

Apply Salting to Reduce Skew:

```
from pyspark.sql.functions import col, rand


df = df.withColumn("salt", (rand() * 10).cast("int"))

df_joined = df_large.join(df_small.withColumn("salt", col("salt")), ["id", "salt"], "inner")
```

- **Salting** helps distribute the load across multiple executors.

7. Reduce Unnecessary Shuffles

- Avoid **excessive groupBy(), distinct(), and repartition()**.

Example:

```
df = df.groupBy("category").agg({"sales": "sum"})  # Optimized Aggregation
```

- Avoid **redundant shuffles** caused by multiple transformations.

8. Caching: Persist Data if Reused Multiple Times

- **Cache** data if it's used repeatedly to avoid recomputation.

Example:

df.cache()

df.count() # Triggers caching

- Useful in scenarios where **multiple actions are performed on the same DataFrame**

.

9. Optimize File Formats & Compression:

- **Use Parquet instead of CSV** for better performance.
- Optimize compression with **Snappy/Zstd**.

Example:

df.write.mode("overwrite").parquet("hdfs://output_folder", compression="snappy")

- **Parquet reduces storage size and speeds up read performance.**